

AQI PREDICTOR

Islamabad Air Quality Prediction System

10Pearls Data Science Project | By Saeed Asif

1. Executive Summary

This report documents the development of a fully automated, serverless Air Quality Index (AQI) prediction system for Islamabad, Pakistan. Built as part of the 10Pearls Data Science project, the system covers the entire machine learning lifecycle from live data ingestion to a publicly accessible prediction dashboard.

The core idea is straightforward: air quality in cities like Islamabad fluctuates significantly across the day and across seasons. Rather than relying on static reports, this system fetches live pollution measurements every hour, processes them through an ML pipeline, and serves real-time AQI predictions along with a 3-day forecast to anyone who visits the dashboard.

The project uses WAQI (World Air Quality Index) as its live data source, Hopsworks as the feature store and model registry, GitHub Actions for automation, and Streamlit for the frontend dashboard. All infrastructure is free-tier and serverless no servers to manage, no manual steps required after deployment.

2. System Architecture

The pipeline follows four sequential stages, each automated to run on a schedule:

Stage	Component	Runs Every	Output
1 — Fetch	fetch.py	1 hour	Raw CSV (staging buffer)
2 — Feature Engineering	features.py	1 hour	Hopsworks Feature Store
3 — Model Training	train.py	1 day	Hopsworks Model Registry
4 — Serving	app.py (Streamlit)	Always on	Live dashboard

GitHub Actions orchestrates stages 1–3 automatically. Streamlit Cloud hosts stage 4 and pulls the latest model from the repository on every reboot.

A deliberate design decision was made to use two separate requirements files: requirements.txt for the Streamlit app (lightweight, no Hopsworks dependency) and requirements-pipeline.txt for the GitHub Actions pipeline (includes Hopsworks and Kafka clients). This prevents version conflicts between the app and pipeline environments.

3. Data

3.1 Live Data — WAQI API

The World Air Quality Index (WAQI) API provides real-time pollution and meteorological readings from monitoring stations across the world. For this project, the Islamabad station is queried every hour. The API returns readings for:

- PM2.5 and PM10 - particulate matter (the primary AQI drivers)
- Carbon Monoxide (CO), Nitrogen Dioxide (NO₂), Sulphur Dioxide (SO₂), Ozone (O₃)
- Temperature, Humidity, Atmospheric Pressure, Wind Speed

Raw readings are appended to a local CSV file (`data/raw/raw_aqi_data.csv`) which acts as a staging buffer. This design means historical raw data accumulates over time without requiring a database.

3.2 Historical Training Dataset

The Pakistan Air Quality Dataset contains 21,840 labelled records spanning multiple years of air quality measurements across Pakistan. Each record includes all ten pollutant and weather features along with an `aqi_category` label derived from PM2.5 values using EPA standard breakpoints.

AQI Category	PM2.5 Range (µg/m³)	Records	Share
Good	0.0 – 12.0	759	3.5%
Moderate	12.1 – 35.4	4,861	22.3%
Unhealthy for Sensitive Groups	35.5 – 55.4	4,199	19.2%
Unhealthy	55.5 – 150.4	8,766	40.1%
Very Unhealthy	150.5 – 250.4	2,617	12.0%
Hazardous	250.5+	638	2.9%

Unhealthy is the most common category (40%), reflecting Pakistan's persistent air quality challenges, particularly in winter months when temperature inversions trap pollutants near the surface.

3.3 Feature Engineering

The features.py pipeline transforms raw sensor readings into 15 model-ready features before storing them in the Hopsworks Feature Store:

Feature Group	Features	Purpose
Raw pollutants	pm2_5, pm10, co, no2, so2, ozone	Direct sensor readings
Meteorological	temperature, humidity, pressure, wind_speed	Weather context
Time-based	hour, day, month, weekday	Captures diurnal and seasonal patterns
Lag features	aqi_lag_1, aqi_lag_2, aqi_lag_3	Recent AQI history
Rolling stats	aqi_rolling_mean_3	3-period smoothed average
Derived	aqi_change	Rate of AQI change

4. Machine Learning Models

4.1 Training Approach

Rather than using a random train/test split, the data was split chronologically - the first 80% of timestamped records for training (17,472 rows) and the most recent 20% for testing (4,368 rows). This time-based split prevents data leakage from future readings into the training set, which is critical for time-series problems.

Three classification models were trained and compared:

- Random Forest - an ensemble of 100 decision trees with max_depth=10 to prevent overfitting
- Gradient Boosting - 100 boosting stages with max_depth=4 for controlled complexity
- Logistic Regression - a linear baseline with StandardScaler preprocessing and L2 regularization

4.2 Performance Results

Model	Accuracy	RMSE	MAE	R ²	Selected?
Random Forest	100.00%	0.0000	0.0000	1.0000	Yes
Gradient Boosting	100.00%	0.0000	0.0000	1.0000	No
Logistic Regression	89.97%	0.5876	0.1777	0.7268	No

4.3 On the 100% Accuracy

The perfect scores for Random Forest and Gradient Boosting are not a sign of overfitting - they reflect the deterministic nature of the dataset. The `aqi_category` labels were generated directly from PM2.5 values using EPA standard breakpoints, which are hard, non-overlapping thresholds. A model that learns these thresholds will always achieve 100% on test data drawn from the same distribution.

Logistic Regression at 90% is the more realistic result - as a linear model, it struggles to learn the sharp step-function boundaries between categories, producing genuine classification errors at borderline PM2.5 values.

Random Forest was selected for production because it correctly implements the threshold rules and produces reliable, interpretable predictions backed by SHAP analysis.

4.4 Feature Importance — SHAP Analysis

SHAP (SHapley Additive exPlanations) was used to explain individual model predictions. The results are scientifically consistent with how AQI is officially calculated:

Feature	SHAP Importance	Interpretation
PM2.5	52.2%	Primary AQI driver - EPA primary standard
PM10	36.4%	Secondary particulate matter - strong contributor
Carbon Monoxide	3.6%	Gaseous pollutant - moderate role
Sulphur Dioxide	3.1%	Gaseous pollutant - moderate role
Nitrogen Dioxide	1.7%	Traffic-related pollutant
Ozone, Humidity, Wind, Pressure, Temperature	3.0%	Combined meteorological effects

PM2.5 and PM10 together account for 88% of feature importance. This is expected - both are the official primary pollutants in EPA and WHO AQI calculations. The remaining 12% from gaseous pollutants and weather shows the model is using all available features, not purely relying on PM2.5.

5. Infrastructure & MLOps

5.1 Hopsworks Feature Store

Engineered features are stored in a Hopsworks Feature Group (aqi_features, version 1) with event_time as the primary temporal key. Due to Kafka connectivity limitations when connecting from external Windows clients, features are uploaded as Parquet files via the Hopsworks Dataset REST API rather than the streaming Kafka interface. This is a valid alternative approach that achieves the same result.

The Feature Store accumulates new feature snapshots every hour as the pipeline runs, building up a growing historical record of Islamabad's air quality that can be used for future model improvements.

5.2 Hopsworks Model Registry

After each training run, the best-performing model (Random Forest) is versioned and registered in Hopsworks Model Registry (aqi_classifier). The registry stores the model artifact, label encoder, feature names list, SHAP summary plot, and evaluation metrics. As of the time of writing, version 10 is the current production version.

5.3 GitHub Actions Automation

Two GitHub Actions workflows handle full pipeline automation:

Workflow	File	Schedule	Steps
Feature Pipeline	fetch_data.yml	Every hour	Fetch API → Engineer features → Push to Hopsworks → Commit CSV
Training Pipeline	train.yml	Daily at 2am UTC	Train models → Evaluate → Register in Hopsworks → Commit models

Both workflows use a git pull --rebase before pushing to handle concurrent commits gracefully. Secrets (API keys) are stored in GitHub repository secrets and injected as environment variables at runtime.

5.4 Streamlit Dashboard

The dashboard (saeed-aqi-predictor.streamlit.app) is deployed on Streamlit Cloud and auto-redeploys on every push to main. Key dashboard sections:

- Hero panel — large AQI number with color-coded category and health advice

- Live metrics — PM2.5, PM10, Humidity, Wind Speed from the latest API reading
- 3-day forecast — 9 time slots (6am, 12pm, 6pm × 3 days) with realistic diurnal pollution patterns
- SHAP feature importance plot — shows which features drive predictions
- Model comparison table — all three models with full metrics
- Health alerts — automatic warnings for Unhealthy/Very Unhealthy/Hazardous conditions

6. Exploratory Data Analysis — Key Findings

A detailed EDA was conducted in notebooks/EDA.ipynb. The main findings:

Diurnal Patterns

PM2.5 concentrations follow a consistent daily cycle. Levels peak in the early morning (6–9am) due to commuter traffic and cooler air that traps pollutants close to the ground. A midday dip occurs as solar heating disperses the boundary layer. A secondary evening peak appears during the 6–9pm rush hour.

Seasonal Patterns

Winter months (November–February) show significantly higher PM2.5 than summer. Temperature inversions during winter prevent vertical mixing, trapping pollutants at ground level. This is a well-documented phenomenon for South Asian cities.

Feature Correlations

PM2.5 and PM10 are moderately correlated ($r \approx 0.72$), as expected — both measure particulate matter from similar sources. CO shows moderate correlation with PM pollutants, consistent with combustion being a shared source. Meteorological features show weaker correlations, confirming their secondary role.

7. Limitations & Future Work

Limitations

- The 3-day forecast uses time-of-day multipliers on current readings rather than a trained time-series model. A proper LSTM or Prophet model would produce more accurate multi-day forecasts.
- Training labels are deterministic from PM2.5 thresholds, which limits the model's ability to handle real-world measurement uncertainty or sensor drift.

- The system currently covers only Islamabad. Extending to Karachi, Lahore, and Peshawar would require city-specific models or a multi-city generalization approach.
- Kafka connectivity from external clients is limited — Hopsworks Feature Group inserts use the Dataset API workaround instead of the native streaming interface.

Future Improvements

- Train an LSTM or Facebook Prophet model for proper time-series forecasting with confidence intervals
- Integrate OpenWeather API alongside WAQI for richer meteorological features (wind direction, cloud cover, UV index)
- Add LIME explanations alongside SHAP for broader model interpretability
- Implement Hopsworks data drift monitoring to detect when the live data distribution shifts from training
- Expand to multiple Pakistani cities and train city-specific or transfer-learning models
- Add SMS/email alerts for hazardous AQI predictions using Twilio or SendGrid

8. Conclusion

This project delivers a complete, production-ready AQI prediction system that meets all requirements from the project brief. Starting from a raw API call, the system automatically engineers features, trains and evaluates multiple ML models, stores everything in a feature store and model registry, runs on a fully automated schedule, and serves predictions through a clean, publicly accessible dashboard.

The architecture choices — WAQI for live data, Hopsworks for MLOps infrastructure, GitHub Actions for CI/CD, Streamlit for serving — were all made with the free-tier constraint in mind while still achieving a genuine production-grade system.

The SHAP analysis confirms that the model's decisions are scientifically grounded, aligning with how official AQI is calculated globally. The 100% accuracy of tree-based models is a feature of the dataset's deterministic labelling, not a red flag, and is explained clearly in the report.

Most importantly, the system works end-to-end automatically — after the initial deployment, no manual intervention is needed. Features update every hour, the model retrains every day, and the dashboard always shows the latest predictions.